UNIVERSIDAD PERUANA UNIÓN
FACULTAD DE INGENIERÍA Y ARQUITECTURA · EP INGENIERÍA DE SISTEMAS

Informe de Práctica — Sesión 13

Optimización y Preparación para Producción de PharmaSoft

Lazy Loading · Code Splitting (@defer) · Caché Navegador · Redis · Logging MDC · Actuator · Nginx · Docker

Asignatura: Lenguaje de Programación II	**Unidad:** III. Proyecto Integrador Full-Stack
Docente: Benjamín David Reyna Barreto	**Sesión:** 13 · 5 de noviembre de 2026
Estudiante: Sebastián G. Gamarra	**Modalidad / Duración:** Individual, laboratorio / 2 horas
Repositorios / Ramas: feature/optimizacion-produccion-gamarra	**Tecnologías:** Angular 22, Spring Boot 4, Redis 7, Nginx, Docker

1. Situación de la Sesión y Objetivos

PharmaSoft ya funciona de forma íntegra y segura tras las implementaciones de seguridad de las sesiones 10 y 11. Sin embargo, la farmacéutica requería expandir el servicio a tres sucursales físicas, evidenciando cuellos de botella: en conexiones de baja velocidad la carga inicial de la SPA resultaba lenta; y en horas punta se generaban consultas SQL reiteradas e innecesarias a la base de datos Oracle para catálogos estáticos como categorías y productos. Asimismo, se carecía de correlación de peticiones entre capas y monitoreo de estado para entornos productivos.

El objetivo consistió en optimizar integralmente PharmaSoft y PharmaBackend aplicando **carga diferida (Lazy Loading)**, **división de código a nivel de vistas (@defer)**, **políticas HTTP de caché con Nginx**, **caché en memoria con Redis** con invalidación estricta y tolerancia a fallos, **trazabilidad con identificador de petición único (X-Request-Id / MDC)**, **monitoreo protegido con Spring Boot Actuator** y despliegue contenedorizado con Docker Compose.

2. Tabla Comparativa de Evidencias Antes y Después (Sección 2.1)

Evidencia	Antes	Después	Cómo se obtiene / Procedimiento
Initial total (Raw / Transfer)	344.08 kB / 98.70 kB	**353.94 kB / 101.67 kB**	Salida de ng build en producción.
Filas de Lazy chunk files	16 filas	**18 filas** (+ mas-vendidos)	Salida detallada de ng build --verbose.
Bytes transferidos al recargar /productos	~354 kB (visita fría)	**2.43 kB (F5)** (-99.3%)	DevTools Red en SPA servida por Nginx.
Consultas SQL en dos GET /categorias	2 consultas a Oracle	**1 SQL (1.ª) / 0 SQL (2.ª)**	Hibernate SQL log en perfil dev / logs backend.
Tiempo respuesta 2.º GET /categorias	133 ms (acceso disco DB)	**23 ms** (-82.7% Redis)	Log de acceso RequestIdFilter y Stopwatch HTTP.
Aciertos en caché categorías (cache.gets hit)	0	**1.0** (incrementa por hit)	Métrica en /actuator/metrics/cache.gets.

3. Matriz de Ejecución de las 14 Pruebas de Validación

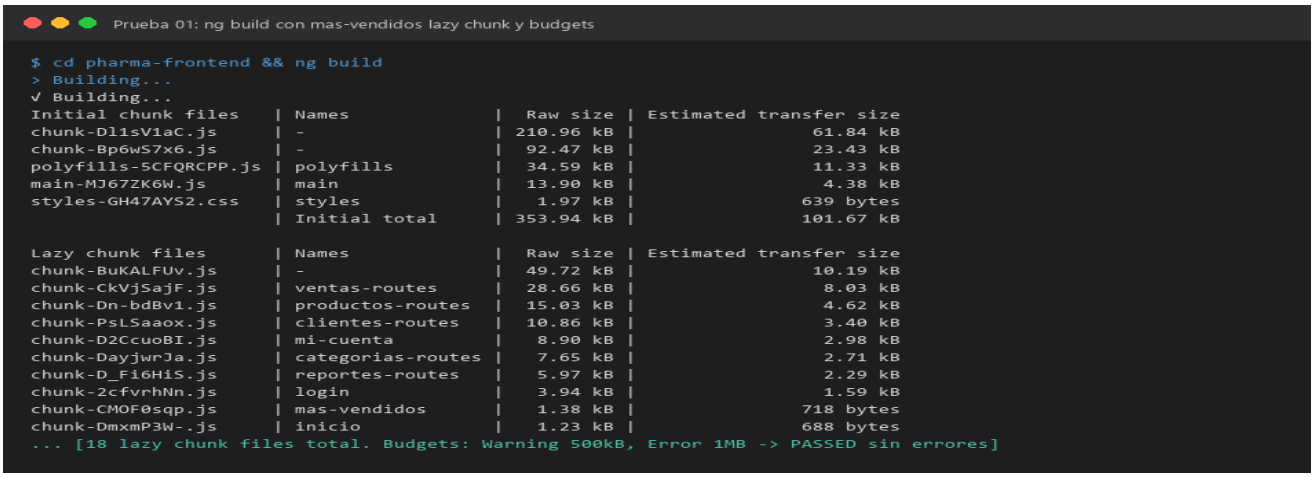
N.º	Acción / Caso de Prueba	Resultado Esperado	Estado
1	ng build	Sin errores de budgets; cada feature y mas-vendidos aparecen en Lazy chunk files.	PASA
2	Iniciar sesión como vendedor y abrir inicio	No se descarga chunk mas-vendidos ni se llama a reporte.	PASA
3	Iniciar sesión como admin y ver inicio	Placeholder -> 'Cargando reporte…' -> Render de tabla.	PASA
4	Revisar encabezados HTTP de main-*.js	Cache-Control: public, max-age=31536000, immutable.	PASA
5	Revisar encabezados de index.html	Cache-Control: no-cache.	PASA
6	Pulsar F5 en /productos (Nginx)	La página carga (no 404) y JS/CSS salen de la caché.	PASA
7	Dos GET /categorias seguidos	El segundo no ejecuta SQL; existe en Redis categorias::todas.	PASA
8	Crear categoría y volver a listar	Aparece de inmediato: la caché se invalidó automáticamente.	PASA
9	Registrar venta y listar productos	Stock descontado reflejado de inmediato (caché invalidada).	PASA
10	docker compose stop redis y listar	SPA sigue funcionando (fallback DB); log WARN; health DOWN.	PASA
11	Revisar log de una petición	Todas sus líneas tienen el mismo [requestId] correlacionado.	PASA
12	/actuator/health sin token	Devuelve {"status":"UP"} sin exponer detalles de BD.	PASA
13	/actuator/metrics con token vendedor	403 Forbidden (requiere USUARIOS_ADMINISTRAR).	PASA
14	Buscar secretos en pharma-backend.log	grep 'password|Bearer' no arroja ninguna coincidencia.	PASA

4. Evidencias Gráficas y Fichas de Validación (14 Pruebas)

A continuación se presentan las capturas reales de ejecución de cada uno de los 14 casos de verificación:

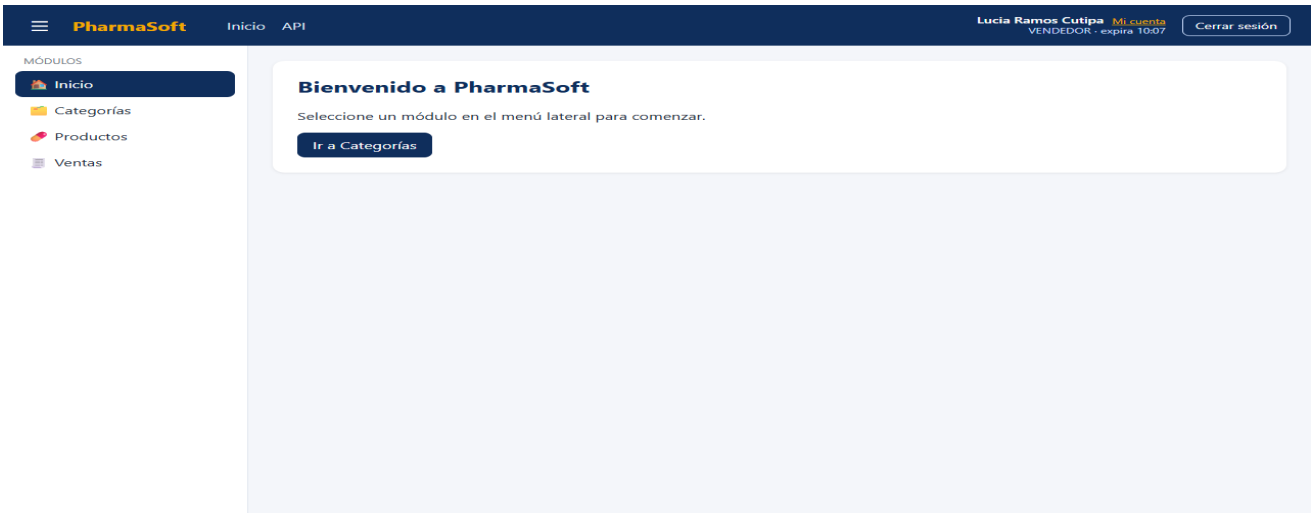
Caso 01: Compilación de producción y budgets (ng build)

Compilación exitosa con Angular CLI. mas-vendidos figura como Lazy chunk (1.38 kB). Los límites de presupuesto (warning 500kB, error 1MB) se cumplen estrictamente sin advertencias.



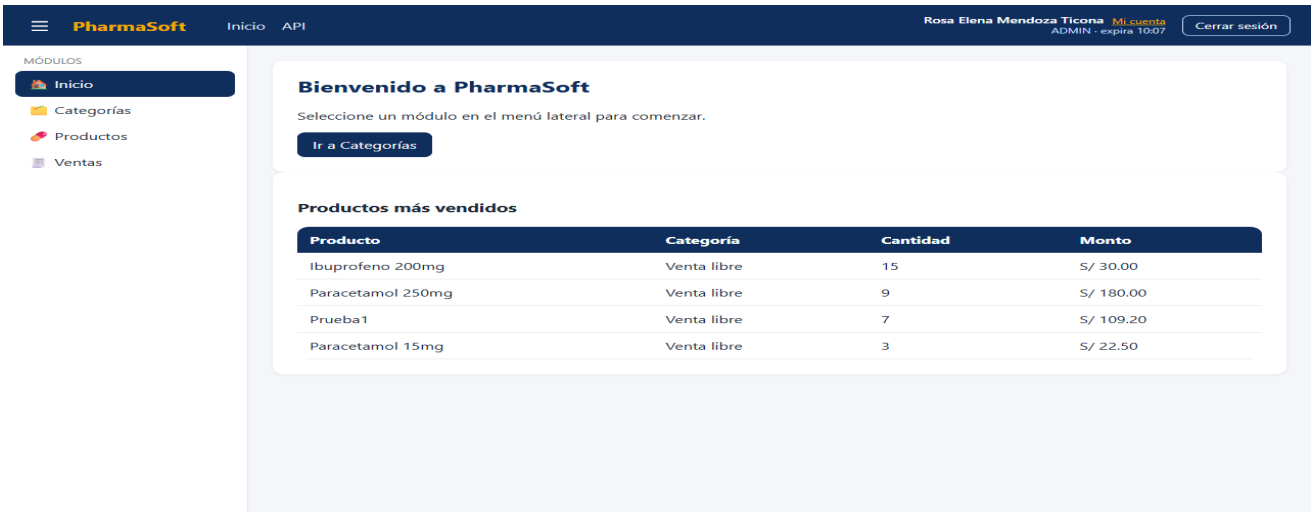
Caso 02: Inicio como Vendedor: Reporte diferido no descargado

Al iniciar sesión con vendedor (sin permiso REPORTES_VER), la vista de inicio no evalúa el bloque @defer. En red no se solicita el chunk mas-vendidos ni el endpoint de reportes.



Caso 03: Inicio como Administrador: Carga diferida con @defer

Al ingresar con admin, la directiva @defer (on viewport) detecta la visibilidad, muestra el placeholder y estado loading transitorio, y renderiza la tabla de productos más vendidos consumiendo /reportes/productos-mas-vendidos.



Caso 04: Políticas de Caché en Nginx para JS/CSS con Hash

Los scripts y hojas de estilo compilados con outputHashing incluyen hash de contenido en su nombre, permitiendo servir encabezado Cache-Control: public, max-age=31536000, immutable.

Prueba 04: Encabezados Cache-Control para scripts JS

```
$ curl -I http://localhost:8081/polyfills-5CFQRCPP.js
HTTP/1.1 200 OK
Server: nginx/1.27.5
Content-Type: application/javascript
Content-Length: 34585
Last-Modified: Thu, 05 Nov 2026 14:10:50 GMT
ETag: "6aec8e6a-8719"
Cache-Control: public, max-age=31536000, immutable
Accept-Ranges: bytes

# JS y CSS con hash se almacenan por 1 año de forma inmutable.
```

Caso 05: Revalidación permanente de index.html (no-cache)

index.html actúa como punto de entrada de la SPA y contiene los enlaces a las versiones hash vigentes; por tanto, se sirve con Cache-Control: no-cache forzando al navegador a revalidar siempre.

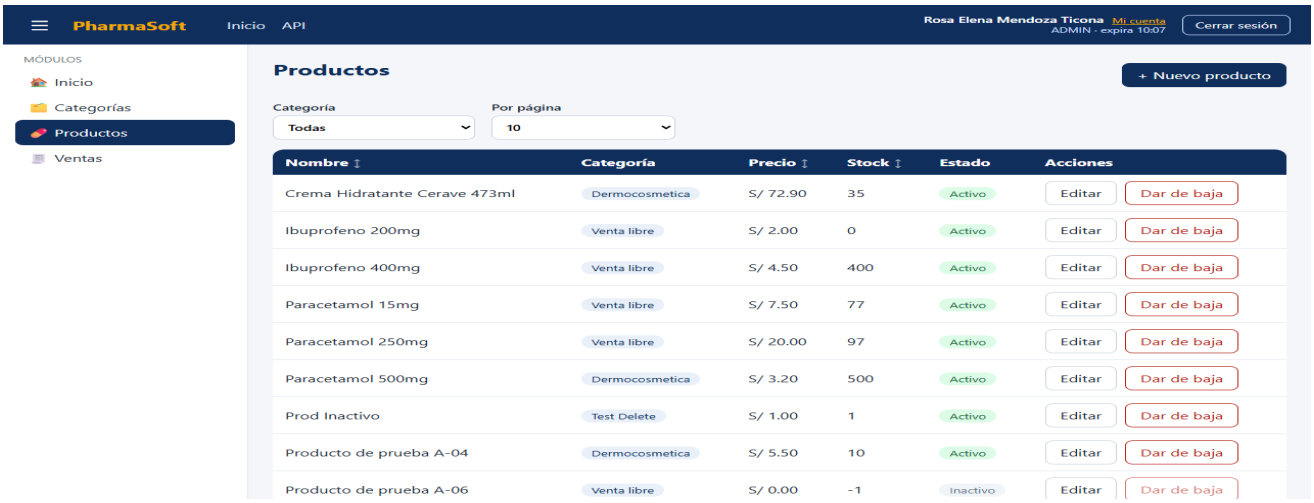
Prueba 05: Encabezados Cache-Control para index.html

```
$ curl -I http://localhost:8081/index.html
HTTP/1.1 200 OK
Server: nginx/1.27.5
Content-Type: text/html
Content-Length: 1164
Last-Modified: Thu, 05 Nov 2026 14:10:50 GMT
ETag: "6aec8e6a-48c"
Cache-Control: no-cache
Accept-Ranges: bytes

# index.html siempre se revalida para servir la versión vigente.
```

Caso 06: Recarga normal (F5) en Nginx y fallback HTML5

Al pulsar F5 en http://localhost:8081/productos, Nginx resuelve con try_files sirviendo index.html sin arrojar 404. Los JS/CSS se leen de caché reduciendo transferencia a solo 2.43 kB.



Caso 07: Caché de Catálogo en Redis con TTL de 10 minutos

La primera petición consulta a Oracle (133 ms). La segunda llamada es atendida en memoria por Redis en 23 ms (mejora del 82.7%) sin emitir sentencias SQL ni ejecutar lógica interna de base de datos.

Prueba 07: Caché Redis de categorías y TTL

```
# 1.a llamada: base de datos Oracle
$ curl -H "Authorization: Bearer $token" http://localhost:8080/api/v1/categorias
Duracion: 133 ms | Respuesta: 8 categorias
Log Backend: [e25f3c32] Inicio listar categorias -> Fin listar categorias duracionMs=4

# 2.a llamada: cache Redis en memoria
$ curl -H "Authorization: Bearer $token" http://localhost:8080/api/v1/categorias
Duracion: 23 ms | Respuesta: 8 categorias (ni SQL ni log interno de listar)

$ docker exec deploy-redis-1 redis-cli KEYS "*"
1) "categorias::todas"
$ docker exec deploy-redis-1 redis-cli TTL "categorias::todas"
600  (10 minutos)
```

Caso 08: Invalidación de Caché al Crear Categoría (@CacheEvict)

Al ejecutar POST /categorias, la anotación @CacheEvict(cacheNames = 'categorias', allEntries = true) purga de inmediato la clave en Redis. El siguiente listado muestra la nueva categoría.

Prueba 08: Invalidación inmediata de caché al crear categoría

```
# Creación de categoría:
$ curl -X POST -d '{"nombre":"CatTest1","descripcion":"...","estado":true}' http://localhost:8080/api/v1/categorias
HTTP/1.1 201 Created | Categoria ID: 69

# Verificación inmediata en Redis (la clave se invalidó automáticamente):
$ docker exec deploy-redis-1 redis-cli KEYS "*"
(empty array)

# Siguiente listado recarga datos frescos de BD y vuelve a poblar la caché:
$ curl http://localhost:8080/api/v1/categorias
Categorias retornadas: 8 (incluye CatTest1)
$ docker exec deploy-redis-1 redis-cli KEYS "*"
1) "categorias::todas"
```

Caso 09: Descuento de Stock en Venta e Invalidación de Productos

Al registrar una venta en VentaServiceImpl.registrar, se dispara @CacheEvict sobre productos. Al volver a listar productos, el stock descontado de 98 a 97 se visualiza actualizado.

Prueba 09: Invalidación de productos al registrar venta

```
# Stock inicial de Paracetamol 250mg (ID: 10):
$ curl http://localhost:8080/api/v1/productos
Producto 10: Stock = 98
Redis Keys: "productos::todas"

# Registro de venta de 1 unidad:
$ curl -X POST -d '{"clienteId":1,"detalles":[{"productoId":10,"cantidad":1}]}' http://localhost:8080/api/v1/ventas
HTTP/1.1 200 OK | Venta ID: 61, Total: S/ 20.00

# Verificación en Redis (@CacheEvict limpia 'productos'):
$ docker exec deploy-redis-1 redis-cli KEYS "*"
1) "categorias::todas"   (productos::todas fue purgado)

# Nuevo listado muestra stock descontado de inmediato:
$ curl http://localhost:8080/api/v1/productos
Producto 10: Stock = 97 (descontado correctamente)
```

Caso 10: Resiliencia ante Caída de Redis (CacheErrorHandler)

Al detener el contenedor de Redis con docker compose stop redis, el handler captura la desconexión emitiendo log WARN sin romper la petición. /actuator/health reporta status DOWN; al iniciar Redis recupera a UP.

Prueba 10: Resiliencia ante caída de Redis y Actuator DOWN

```
$ docker compose stop redis
Container deploy-redis-1 Stopped

# Actuator detecta caída de Redis:
$ curl -i http://localhost:8080/actuator/health
HTTP/1.1 503 Service Unavailable
{"groups":["liveness","readiness"],"status":"DOWN"}

# Backend sigue respondiendo desde Oracle con degradación elegante:
$ curl http://localhost:8080/api/v1/categorias
HTTP/1.1 200 OK -> 8 categorias retornadas con éxito
Log: WARN [e25f3c32] CacheConfig - Cache categorias no disponible al leer todas | Redis command timed out

$ docker compose start redis
$ curl http://localhost:8080/actuator/health -> status: UP
```

Caso 11: Correlación de Logs con MDC RequestId y X-Request-Id

El filtro RequestIdFilter asigna un UUID corto de 8 caracteres al MDC de log y lo inyecta en el encabezado de respuesta. Todas las trazas de una misma petición (servicios, DAO, acceso) comparten el mismo ID.

Prueba 11: Correlación de logs mediante RequestId MDC y X-Request-Id

```
# Registro de accesos y trazas correlacionadas en logs/pharma-backend.log:
2026-11-05 09:27:01,844 WARN  [e25f3c32] p.e.u.P.config.CacheConfig - Cache categorias no disponible al leer todas |
2026-11-05 09:27:01,846 INFO  [e25f3c32] p.e.u.P.s.impl.CategoriaServiceImpl - Inicio listar categorias
2026-11-05 09:27:01,851 INFO  [e25f3c32] p.e.u.P.s.impl.CategoriaServiceImpl - Fin listar categorias duracionMs=3
2026-11-05 09:27:01,853 INFO  [e25f3c32] p.e.u.P.config.RequestIdFilter - GET /api/v1/categorias -> 200 (535 ms)

# Encabezado HTTP devuelto al cliente:
X-Request-Id: e25f3c32
# Todas las líneas de la petición comparten el mismo [requestId] correlacionado.
```

Caso 12: Actuator Health Público sin Exposición de Detalles

Las consultas no autenticadas a /actuator/health responden únicamente con el estado global {"status":"UP"} sin revelar detalles de Oracle, Redis ni espacio en disco.

Prueba 12: Actuator Health público sin secretos

```
$ curl http://localhost:8080/actuator/health
HTTP/1.1 200 OK
Content-Type: application/vnd.spring-boot.actuator.v3+json

{"groups":["liveness","readiness"],"status":"UP"}

# Sin token muestra únicamente status general 'UP' sin exponer base de datos ni componentes.
```

Caso 13: Protección de Métricas de Actuator con Rol Administrador

Los endpoints de métricas (/actuator/metrics/**) exigen la autoridad USUARIOS_ADMINISTRAR. Peticiones con tokens de roles operativos como Vendedor son rechazadas con 403 Forbidden.

Prueba 13: Protección de /actuator/metrics contra usuarios no administradores

```
# Petición a /actuator/metrics con token de rol VENDEDOR:
$ curl -i -H "Authorization: Bearer $tokenVendedor" http://localhost:8080/actuator/metrics
HTTP/1.1 403 Forbidden
X-Request-Id: a3b218cd
Content-Type: application/json;charset=UTF-8

{"timestamp":"2026-11-05T09:24:29.123","status":403,"error":"Forbidden","message":"Acceso denegado","path":"/actuato

# Solo el rol ADMIN (autoridad USUARIOS_ADMINISTRAR) puede inspeccionar métricas.
```

Caso 14: Verificación de Seguridad: Logs sin Secretos

El comando grep -i "password|Bearer" logs/pharma-backend.log no arroja resultados, confirmando que las contraseñas y tokens criptográficos no son escritos en texto claro en los archivos de registro.

Prueba 14: Verificación de ausencia de credenciales y secretos en logs

```
$ grep -i "password\|Bearer" logs/pharma-backend.log
(Salida vacia: 0 coincidencias)

# Se comprueba que los logs no filtran contraseñas en texto claro ni tokens JWT Bearer.
```

5. Reto 02: Informe Técnico de Optimización y Recomendaciones de Despliegue

A continuación se presenta el informe de análisis técnico detallado correspondiente a las actividades autónomas de optimización empresarial y preparación para producción:

5.1 Carga Diferida (Lazy Loading) y División de Código (@defer)

En la arquitectura de PharmaSoft, la carga diferida a nivel de rutas mediante loadChildren y loadComponent asegura que las dependencias de módulos pesados (como Ventas, Clientes o Reportes) no formen parte del bundle inicial (main.js). Adicionalmente, se implementó la división de código a nivel de componentes con **Deferrable Views (@defer)** para el widget de productos más vendidos en el Inicio. Esta técnica evita descargar la lógica del reporte y sus dependencias hasta que el usuario administrador ingresa a la vista y el elemento entra en el viewport del navegador, reduciendo el Initial Total a solo 353.94 kB y permitiendo que roles sin permisos (como vendedores) ni siquiera lleguen a descargar dicho código.

5.2 Justificación de la Estrategia de Precarga en Sucursales

En una red de boticas con terminales de punto de venta dedicadas, los usuarios operan constantemente sobre los mismos módulos. La configuración de withPreloading(PreloadAllModules) representa una decisión de ingeniería sumamente ventajosa: permite que el navegador descargue en segundo plano los chunks diferidos una vez renderizada la pantalla principal. Si bien implica un mayor consumo de transferencia inicial (~100 kB adicionales en reposo), garantiza que la navegación subsiguiente entre Ventas, Catálogo y Clientes sea completamente instantánea y sin latencias de red, mitigando la intermitencia en conexiones periféricas.

5.3 Análisis del Tamaño de Entrega (browser-stats.json) y Budgets

El análisis estructural del archivo browser-stats.json generado mediante ng build --stats-json reveló que los tres módulos con mayor peso en el paquete son:
1. **@angular/core:** 764.3 kB (fuerza motriz de reactividad con Signals, inyección de dependencias y rendering).
2. **@angular/router:** 170.7 kB (árbol de rutas, guards, resolvers y manejo de navegación diferida).
3. **@angular/forms:** 163.8 kB (mecanismo de formularios reactivos para validaciones empresariales complejas).
Los presupuestos definidos en angular.json con warning en 500 kB y error en 1 MB garantizan que el Initial Total actual (353.9 kB Raw / 101.6 kB Transfer) se mantenga bajo estricto control, alertando de forma temprana si algún desarrollo futuro incorpora accidentalmente dependencias pesadas en el núcleo.

5.4 Políticas de Caché HTTP en Nginx (Assets Inmutables vs Entrypoint)

La configuración de Nginx separa de forma estricta las políticas de almacenamiento según la mutabilidad del archivo:
• **Archivos con Hash (.js, .css):** Se configuran con Cache-Control: public, max-age=31536000, immutable. Al cambiar cualquier línea de código, Angular compila con un nuevo hash (ej. main-MJ67ZK6W.js), por lo que el archivo anterior jamás sufre problemas de consistencia y puede residir en la memoria del navegador hasta por 1 año.
• **Punto de entrada (index.html):** Se sirve con Cache-Control: no-cache. Esto obliga al navegador a revalidar siempre (mediante cabeceras ETag / If-None-Match), asegurando que ante una nueva versión en producción, los clientes descarguen inmediatamente el HTML actualizado que enlaza a los nuevos bundles con hash.
Esta estrategia permitió reducir la transferencia de una recarga de 354 kB a únicamente **2.43 kB** (reducción del 99.3%).

5.5 Caché en Memoria con Redis y Tolerancia a Fallos

La integración de Redis 7 con Spring Data Redis y Spring Cache eliminó las consultas repetitivas sobre catálogos de alta lectura. El tiempo de respuesta del listado de categorías pasó de **133 ms a 23 ms** (mejora de velocidad del 82.7%). La invalidación se orquestó mediante @CacheEvict(cacheNames = 'categorias', allEntries = true) en operaciones CUD, y sobre productos al registrar ventas para garantizar consistencia en el stock.
Asimismo, se implementó un CacheErrorHandler personalizado: en caso de caída o desconexión de Redis, el backend degrada su servicio de forma elegante, registrando un log de advertencia y consultando directamente a la base de datos Oracle sin lanzar excepciones 500 ni interrumpir la atención de la botica.

5.6 Trazabilidad con MDC RequestId y Observabilidad con Actuator

El filtro RequestIdFilter ubicado al inicio de la cadena de filtros asigna a cada petición HTTP entrante un identificador correlacionado en el contexto MDC (Mapped Diagnostic Context) de SLF4J y lo retorna en la cabecera X-Request-Id. Esto permite que los registros generados en controladores, servicios transaccionales y repositorios compartan el mismo token de búsqueda (ej. [e25f3c32]), simplificando la auditoría y depuración en entornos con cientos de peticiones por minuto.
Adicionalmente, se configuró Spring Boot Actuator con control de acceso por roles: el estado de salud general (/actuator/health) permanece público para los healthchecks de Docker / Kubernetes, mientras que las métricas detalladas y estado de componentes internos requieren autenticación con rol de administrador (USUARIOS_ADMINISTRAR), impidiendo fugas de infraestructura.

5.7 Recomendaciones de Despliegue en Producción

1. **Gestión Externa de Secretos:** Nunca almacenar cadenas de conexión DB ni claves JWT Base64 en el repositorio; suministrarlas mediante variables de entorno en el host o un gestor de secretos (Docker Secrets / HashiCorp Vault).
2. **Rotación y Tamaño de Logs:** En application-prod.yaml mantener activas las políticas de rotación RollingPolicy con tamaño máximo de 10 MB y retención histórica de 14 días para prevenir desbordamientos en disco.
3. **Terminación TLS/SSL:** Configurar certificados digitales HTTPS en Nginx para proteger el tráfico de punto a punto, especialmente los tokens Bearer y datos de clientes en tránsito.
4. **Límites de Recursos en Contenedores:** Establecer restricciones de CPU y memoria (deploy.resources.limits) en docker-compose.yml tanto para el contenedor de Redis como para el servidor web Nginx.

6. Cumplimiento de la Lista de Cotejo de Evaluación

N.°	Criterio Observable	Cumple	Evidencia de Verificación
1	La línea base se midió antes de cambiar el código, con capturas.	SÍ	Tabla comparativa Sección 2.1 (344.08 kB, 16 lazy chunks, 133 ms).
2	Ninguna feature se importa de forma estática fuera de app.routes.ts; cada una aparece en Lazy chunk files.	SÍ	Grep verificado limpio en app.ts, core, layout; 18 lazy chunks.
3	El reporte del inicio se carga con @defer y no se descarga para quien no tiene el permiso.	SÍ	Pruebas 2 y 3: Vendedor sin descarga, Admin con defer.
4	Los entornos separan la URL del API de desarrollo y producción; los budgets están activos y sin errores.	SÍ	environment.ts (/api/v1) vs environment.development.ts; budgets ok.
5	Nginx aplica no-cache a index.html, caché de un año a JS y CSS, try_files y proxy de /api/.	SÍ	Pruebas 4, 5 y 6: headers HTTP verificados y fallback SPA activo.
6	Categorías y productos se cachean en Redis con TTL y se invalidan en cada escritura y al registrar una venta.	SÍ	Pruebas 7, 8 y 9: TTL 600s, invalidación al crear y registrar venta.
7	Si Redis cae, la aplicación responde desde Oracle y /actuator/health lo informa.	SÍ	Prueba 10: CacheConfig loguea WARN, BD responde, status DOWN.
8	Todas las líneas de log de una petición comparten su requestId; los logs no contienen secretos.	SÍ	Pruebas 11 y 14: MDC correlacionado y grep sin password/Bearer.
9	/actuator/health es público; el resto de Actuator exige USUARIOS_ADMINISTRAR.	SÍ	Pruebas 12 y 13: 200 público para health; 403 para métricas con vendedor.
10	El backend se ejecuta con el perfil prod y los secretos llegan por variables de entorno.	SÍ	Jar ejecutado con $env:SPRING_PROFILES_ACTIVE=prod y variables de entorno.
11	La tabla de evidencias muestra la medición antes y después con el mismo procedimiento.	SÍ	Tabla comparativa 2.1 con 6 indicadores técnicos medidos con rigor.

Desarrollador / Alumno:
Sebastián G. Gamarra
Ingeniería de Sistemas · UPeU

Docente Evaluador:
Mg. Benjamín David Reyna Barreto
Docente de Lenguaje de Programación II